

Модуль 2. “Компілятори та предметно-орієнтовані мови: загальні концепції”

дисципліни

“Розробка компіляторів для предметно-орієнтованих мов”

Лектор: *проф. Григорій ЖОЛТКЕВИЧ, д.т.н.*

Зміст

1	Що таке компілятор?	2
1.1	Етапи компіляції	2
1.2	Взаємозв’язок між етапами компіляції	3
2	Простий приклад	4
2.1	Визначення мови	4
2.2	Лексичний аналіз	4
2.3	Синтаксичний аналіз	6
2.4	Трансляція	8
3	Особливості процесу компіляції для предметно-орієнтованих мов	11
4	Висновки	12
	Література	14

Тема “Компілятори та предметно-орієнтовані мови: загальні концепції” має на меті надати загальне уявлення про компілятор та процес компіляції, а також уточнити поняття предметно-орієнтованої мови в контексті компіляції. Насамперед, компілятор визначається як програма, що перетворює (компілює) код, написаний однією мовою (вихідною), у код, написаний іншою мовою (цільовою). Зазвичай це означає трансформацію високо-рівневої мови (наприклад, C++, Java, Python) у низькорівневий код, виконуваний здатний комп’ютер.

Процес компіляції є багатоетапним і складається з кількох ключових стадій, які взаємопов’язані через структури даних:

- Лексичний аналіз (розпізнавання слів) є першим етапом, на якому вихідний код перетворюється на послідовність значущих одиниць – токенів. За розпізнавання токенів відповідає лексичний аналізатор (сканер), який використовує регулярні вирази як патерни для розпізнавання. На цьому етапі ігноруються несуттєві символи, як-от пробіли, та виявляються лексичні помилки.
- Синтаксичний аналіз (перевірка граматики) є другим ключовим етапом, на якому перевіряється відповідність послідовності токенів граматичним правилам мови. Цей етап виконується парсером, який, використовуючи контекстно-вільні граматики, будує абстрактне синтаксичне дерево (AST) – ієрархічне представлення структури програми. На цій стадії виявляються синтаксичні помилки. Для складних мов можуть застосовуватися генератори парсерів, тоді як для простих предметно-орієнтованих мов підходить ліво-рекурсивний спадний парсинг.
- Семантичний аналіз (перевірка логіки) є третім (опціональним) етапом, який перевіряє логічну коректність програми, включаючи сумісність типів, правильність використання змінних та визначеність функцій. Особливо для предметно-орієнтованих мов, семантичний аналіз дозволяє проводити валідацію на доменному рівні, зменшуючи кількість логічних помилок.
- Трансляція (генерація цільового коду) є останнім ключовим етапом, метою якого є генерація виконуваного цільового коду – машинного коду, байт-коду або коду іншої мови програмування. У контексті предметно-орієнтованих мов, цей етап може включати синтез застосунків з програмних компонентів.

Для ілюстрації цих етапів наводиться простий приклад компілятора для мови арифметичних виразів, демонструючи, як вихідний код перетворюється на послідовність токенів, потім на AST, і зрештою – на послідовність команд для стекового обчислювача.

Особлива увага приділена відмінностям процесу компіляції для предметно-орієнтованих мов порівняно з мовами загального призначення. Предметно-орієнтовані мови характеризуються спрощеністю та вузькою спеціалізацією, що призводить до менш складних граматики та спрощених етапів аналізу. Компілятори предметно-орієнтованих мов рідко виконують складні оптимізації, оскільки це часто делегується цільовій платформі (наприклад, СУБД для SQL-запитів). Специфічний характер трансляції для таких мов полягає в перетворенні коду на іншу високорівневу мову (Python, C++, JavaScript), XML, SQL-запити або виклики методів існуючих бібліотек, а також у можливості безпосередньої інтерпретації через шаблон "відвідувач". Розробка компіляторів для предметно-орієнтованих мов часто прискорюється за рахунок використання генераторів парсерів (наприклад, ANTLR, Tree-sitter) та тісної інтеграції з існуючим кодом.

Таким чином, розуміння етапів компіляції та їх адаптація до специфіки предметно-орієнтованих мов є критично важливим для створення ефективних та цілеспрямованих інструментів розробки, які мінімізують складність і зосереджуються на перетворенні вузькоспеціалізованої логіки на виконуваний інструкції або дані.

1 Що таке компілятор?

Коротко кажучи, компілятор є програмою, яка перетворює (компілює) код, написаний однією мовою, у код, написаний іншою мовою. Зазвичай, це перетворення програми, написаної мовою програмування високого рівня (наприклад, C++, Java або Python) у низькорівневий код, який може виконати комп'ютер.

Цей процес не є миттєвим. Він складається з кількох ключових етапів.

1.1 Етапи компіляції

Лексичний аналіз (розпізнавання слів). Цей етап компіляції є її першим ключовим етапом.

Лексичний аналізатор є програмою, що відповідає за розпізнавання “слів” у коді, які називають токенами. Для цього використовується техніка *регулярних виразів* та *скінченних акцепторів*.

Кожен розпізнаний токен представляється об'єктом, що містить його тип (наприклад, KEYWORD_IF), значення (наприклад, 'if' та позицію у вихідному коді – номер рядка, номер стовпця).

Лексичний аналізатор також ігнорує певні символи, такі як пробіли та переноси рядків, якщо вони не є значущими для логіки мови.

Якщо під час цього етапу зустрічається невідомий символ, він генерує лексичну помилку.

Таким чином, в результаті лексичного аналізу вихідний код перетворюється на послідовність значущих (таких, що не ігноруються) токенів.

Синтаксичний аналіз (перевірка граматики). Цей етап компіляції є її другим ключовим етапом.

На цьому етапі перевіряється, чи відповідає послідовність токенів граматичним правилам мови.

При проєктуванні синтаксичних аналізаторів (парсерів) для визначення синтаксичної структури мови використовують *породжуючі граматики Хомського*, а точніше *контекстно-вільні граматики*, з чим ми познайомимося в цьому курсі.

Завданням парсеру є побудова абстрактного синтаксичного дерева (Abstract Syntax Tree, AST), яке є ієрархічним представленням структури програми.

Для складних формальних мов можуть використовуватися сучасні генератори парсерів, такі як Tree-sitter [1], ANTLR [2] або PEG-парсери [3], які забезпечують надійну обробку складних граматик та функцій, таких як підсвічування синтаксису та інкрементний парсинг.

Для простих предметно-орієнтованих мов добре підходить ліво-рекурсивний спадний парсинг, з яким ми познайомимося у цьому курсі.

На етапі парсингу виявляються синтаксичні помилки, якщо код не відповідає граматиці мови.

Семантичний аналіз (перевірка логіки). Це третій ключовий етап компіляції, що відбувається після лексичного та синтаксичного аналізу.

На цьому етапі перевіряється логічна коректність програми, тобто її змістовне значення.

Семантичний аналізатор перевіряє такі аспекти, як сумісність типів, правильність використання змінних, визначеність функцій тощо. Для предметно-орієнтованих мов семантичний аналіз дозволяє здійснювати валідацію також на доменному рівні, що є значною перевагою.

Якщо конструкції мови є безпечними, то будь-яке речення, написане з їх використанням, можна вважати безпечним, що допомагає зменшити логічні помилки, які часто виникають у мовах загального призначення через необхідність ручної перевірки доменних правил.

Наприклад, для мови SQL, інтерпретатори перевіряють семантичну коректність запитів, щоб переконатися, що їх виконання завершується і вони є “безпечними”. Існує властивість запиту “стратифікованість”, яка є розв’язною та достатньою для безпечності запиту [4]. Вона використовується в більшості реляційних СУБД для перевірки безпечності запиту.

На цьому етапі компіляції виявляються семантичні помилки, які стосуються невідповідності логіці або правилам предметної області. Повідомлення про помилки повинні бути зрозумілими для доменних експертів і пропонувати можливі виправлення.

Трансляція (генерація цільового коду). Це четвертий і останній ключовий етап компіляції.

Основною метою є генерація цільового коду, який може бути виконаний комп’ютером. Це може бути низькорівневий код, машинний код, байт-код або код іншої мови програмування.

У контексті розробки програмного забезпечення, особливо для предметно-орієнтованих мов, цей етап може включати синтез відповідних застосунків з програмних компонентів.

1.2 Взаємозв’язок між етапами компіляції

Маючи опис етапів трансляції давайте подивимося через які структури даних вони пов’язані між собою у єдиний процес, чим ці етапи керуються (див. Табл. 1).

Табл. 1: Етапи компіляції

Етап компіляції	приймає	використовує	генерує
1	2	3	4
<i>Лексичний аналіз</i> виконується лексичним аналізатором (сканером, лексером)	Код написаний вихідною мовою	Правила розпізнавання токенів мови	Послідовність токенів
<i>Синтаксичний аналіз</i> виконується синтаксичним аналізатором (парсером)	Послідовність токенів	Граматичні правила вихідної мови	Абстрактне синтаксичне дерево (АСД) вихідного коду
<i>Семантичний аналіз</i> виконується (опціонально) семантичним аналізатором	Абстрактне синтаксичне дерево (АСД) вихідного коду	Семантичні обмеження предметної області	Звіт про семантичні помилки
<i>Трансляція</i> виконується транслятором	Абстрактне синтаксичне дерево (АСД) вихідного коду	Правила трансляції	Код цільовою мовою, що відповідає вихідному коду

Найбільш важливим є стовпчик 3 у Табл. 1. Саме аналіз цього стовпчика дозволяє поставити принципові питання, відповідь на які дозволяє створити автоматизовану технологію розробки компіляторів, в тому числі й для предметно-орієнтованих мов. Цими питаннями є наступні.

1. Як представляти правила розпізнавання токенів та використовувати їх?
2. Як формулювати правила граматики та застосовувати їх в процесі парсингу?

3. Які семантичні властивості можна перевіряти та як це робити?
4. Як специфікувати правила генерації цільового коду за абстрактним синтаксичним деревом вихідного коду?

Для відповіді на ці питання зазвичай використовується теорія формальних мов.

2 Простий приклад

Для ілюстрації етапів компіляції та особливостей розробки компіляторів для предметно-орієнтованих мов розглянемо простий приклад компілятора для мови арифметичних виразів.

Ця мова дозволяє створювати вирази, які складаються з цілих констант та складених виразів. Складені вирази мають форму (operation operand operand), де operation може бути + або *, а operand – це інший арифметичний вираз. Прикладом виразу в цій мові є

$$(+ (* 3 10) 5)$$

2.1 Визначення мови

Для опису цієї мови ми використовуємо PEG-граматику [3]:

```
Expression <- Constant / ComposedExpression
Constant <- '0' / [1-9] [0-9]*
ComposedExpression <- '(' Operation Expression Expression ')'
Operation <- '+' / '*'
```

Етапи компіляції для цього простого випадку включають лексичний аналіз, синтаксичний аналіз та трансляцію (генерацію цільового коду).

2.2 Лексичний аналіз

Першим етапом компіляції є лексичний аналіз, який перетворює вхідний рядок коду на послідовність значущих одиниць, що називаються токенами. Для нашої мови арифметичних виразів визначено наступні лексичні класи токенів та регулярні вирази, що є правилами розпізнавання (див. Табл. 2).

Табл. 2: Лексичні класи мови арифметичних виразів

Лексичний клас	Назва	Регулярний вираз
LPAREN	відкрита дужка	<code>r'\('</code>
RPAREN	закрита дужка	<code>r'\)' '</code>
PLUS	операція додавання	<code>r'\+' '</code>
MULT	операція множення	<code>r'*' '</code>
NUMBER	число	<code>r'0 [1-9] [0-9]*'</code>

Для представлення регулярних виразів в таблиці використано мову регулярних виразів з Python-бібліотеки `re` (деталі дивись у [5]).

Програмно, Табл. 2 може бути представлена як словник `LEXICAL_CLASSES`, ключами якого є назви лексичних класів, а значеннями – регулярні вирази, що задають правила розпізнавання токенів.

```

LEXICAL_CLASSES = {
    'LPAREN': r'\(',
    'RPAREN': r'\)',
    'PLUS': r'\+',
    'MULT': r'\*',
    'NUMBER': r'0[1-9][0-9]*'
}

```

Лістинг 1: Лексичні класи

Кожен розпізнаний токен представляється як об'єкт класу `Token`, який містить його тип (`type: str`, наприклад, `'NUMBER'`) та значення (`value: str`, наприклад, `'123'`). При створенні об'єкта `Token`, відбувається валідація, яка перевіряє, чи значення токена відповідає його визначеному регулярному виразу.

```

@dataclass(frozen=True)
class Token:
    type: str
    value: str

    def __post_init__(self):
        """
        Перевіряє, чи відповідає значення токена його типу,
        використовуючи словник LEXICAL_CLASSES.
        """
        global LEXICAL_CLASSES
        if self.type not in LEXICAL_CLASSES:
            raise ValueError(f"Невідомий тип токена: {self.type}")
        pattern = LEXICAL_CLASSES[self.type]
        if not re.fullmatch(pattern, self.value):
            raise ValueError(
                f"Некоректне значення '{self.value}' для типу токена "
                f"'{self.type}'. Очікувався патерн: {pattern}"
            )

```

Лістинг 2: Клас `Token`

Сам лексичний аналізатор реалізовано як клас `Lexer`.

```

class Lexer:
    def __init__(self, text: str):
        self.text = text
        self.pos = 0

    def get_token(self) -> Optional[Token]:
        """Повертає наступний токен з вхідного рядка."""
        self.skip_whitespace()
        if self.pos >= len(self.text):
            return None # Кінець рядка
        # Пошук першого відповідного патерна
        for token_type, pattern in LEXICAL_CLASSES.items():
            match = re.match(pattern, self.text[self.pos:])
            if match:
                value = match.group(0)
                self.pos += len(value)

```

```

        return Token(token_type, value)
    raise ValueError(
        f"Невідомий токен на позиції {self.pos}: "
        f"{self.text[self.pos :]}"
    )

    def skip_whitespace(self):
        """Пропускає пробіли в рядку. """
        match = re.match(r'\s+', self.text[self.pos:])
        if match:
            self.pos += len(match.group(0))

    def tokenize(self) -> list[Token]:
        """Виконує повний лексичний аналіз і повертає список токенів. """
        tokens = []
        while True:
            token = self.get_token()
            if token is None:
                break
            tokens.append(token)
        return tokens

```

Лістинг 3: Клас Lexer

Виконання лексичного аналізу для виразу "(+ (* 3 10)) 5" дасть таку послідовність токенів

```

Token(type='LPAREN', value='(')
Token(type='PLUS', value='+')
Token(type='LPAREN', value='(')
Token(type='MULT', value='*')
Token(type='NUMBER', value='3')
Token(type='NUMBER', value='10')
Token(type='RPAREN', value=')')
Token(type='NUMBER', value='5')
Token(type='RPAREN', value=')')

```

2.3 Синтаксичний аналіз

Отримана послідовність токенів передається на етап синтаксичного аналізу, де перевіряється її відповідність граматичним правилам мови. Завданням парсера є побудова абстрактного синтаксичного дерева (АСД) (Abstract Syntax Tree, AST), яке є ієрархічним представленням структури програми. В нашій мові арифметичних виразів, основні компоненти виразу представлені класами Expression (як базовий), Constant (для цілих чисел), Plus (для додавання) та Mult (для множення).

```

class Expression:
    pass

    @dataclass(frozen=True)
    class Constant(Expression):
        value: int

    @dataclass(frozen=True)

```

```

class Plus(Expression):
    left: Expression
    right: Expression

@dataclass(frozen=True)
class Mult(Expression):
    left: Expression
    right: Expression

```

Лістинг 4: Структура AST

Парсер, реалізований як клас `Parser`, використовує методи, такі як

- `consume` для перевірки та переходу до наступного токена, та
- рекурсивні функції `parse_constant`, `parse_expression`, `parse_composed_expression` для розбору відповідних граматичних конструкцій.

```

class Parser:
    def __init__(self, tokens: List[Token]):
        self.tokens = tokens
        self.pos = 0

    def current_token(self) -> Optional[Token]:
        """Повертає поточний токен або None, якщо досягнуто кінця. """
        if self.pos < len(self.tokens):
            return self.tokens[self.pos]
        return None

    def consume(self, expected_type: str) -> Token:
        """Перевіряє тип поточного токена та переходить до наступного. """
        token = self.current_token()
        if token is None or token.type != expected_type:
            raise SyntaxError(
                f"Очікувався токен типу '{expected_type}', "
                f"але отримано {token}"
            )
        self.pos += 1
        return token

    def parse_constant(self) -> Constant:
        """Парсить константу. """
        token = self.consume('NUMBER')
        return Constant(value=int(token.value))

    def parse_expression(self) -> Expression:
        """Головний метод, що розбирає будьякий- вираз. """
        token = self.current_token()
        if token is None:
            raise SyntaxError("Неочікуваний кінець вхідних даних")
        if token.type == 'NUMBER':
            return self.parse_constant()

```



```

elif token.type == 'LPAREN':
    return self.parse_composed_expression()
else:
    raise SyntaxError(f"Неочікуваний токен: {token}")

def parse_composed_expression(self) -> Expression:
    """Парсить складений вираз. """
    self.consume('LPAREN') # Приймаємо '('
    token = self.current_token()
    if token.type == 'PLUS':
        self.consume('PLUS')
        left = self.parse_expression()
        right = self.parse_expression()
        self.consume('RPAREN') # Споживаємо ')'
        return Plus(left, right)
    elif token.type == 'MULT':
        self.consume('MULT')
        left = self.parse_expression()
        right = self.parse_expression()
        self.consume('RPAREN') # Споживаємо ')'
        return Mult(left, right)
    else:
        raise SyntaxError(
            f"Очікувалась операція, але отримано {token}")

def parse(self) -> Expression:
    """Головний метод, що запускає парсинг. """
    ast = self.parse_expression()
    if self.current_token() is not None:
        raise SyntaxError(
            f"Залишилися зайві токени після парсингу: "
            f"{self.current_token()}")
    return ast\end{minted}

```

Лістинг 5: Клас Token

З отриманої раніше послідовності токенів метод `Parser.parse()` буде наступне абстрактне синтаксичне дерево

```

Plus(left=Mult(left=Constant(value=3),
               right=Constant(value=10)),
     right=Constant(value=5))

```

2.4 Трансляція

Трансляція (генерація цільового коду) є останнім ключовим етапом компіляції, мета якого – генерація виконуваного цільового коду. У контексті предметно-орієнтованих мов, це часто означає перетворення коду на іншу високорівневу мову, XML, SQL-запити або виклики методів в існуючих бібліотеках.

В цьому прикладі ми визначаємо простий стековий обчислювач, який виконує арифметичні операції, використовуючи стек - список, що працює за принципом LIFO (Last In, First Out), для зберігання проміжних результатів.

Обчислювач підтримує дві основні команди:

- **save**: проштовхує число в стек та
- **eval**: виконує операцію (додавання або множення) з двома верхніми елементами стека, видаляючи їх та натомість проштовхуючи в стек результат виконання операції.

Обчислювач реалізований класом `StackCalculator`, що наведений нижче.

```
class StackCalculator:
    """
    Простий стековий обчислювач, що виконує операції, використовуючи стек.
    """
    def __init__(self):
        # Стек як список Python
        self._stack = []

    def save(self, number: int):
        """Провтовхує число на стек."""
        self._stack.append(number)

    def eval(self, operation: str):
        """
        Виконує операцію з двома верхніми елементами стека.
        У випадку недостатньої кількості елементів на стеку,
        буде викликана помилка ValueError.
        """
        if len(self._stack) < 2:
            raise ValueError(
                "Помилка! На стеку недостатньо елементів для виконання "
                "операції 'eval'. Необхідно щонайменше 2 елементи.")
        # Видаляємо два верхні елементи
        operand2 = self._stack.pop()
        operand1 = self._stack.pop()
        result = 0
        if operation == "plus":
            result = operand1 + operand2
        elif operation == "mult":
            result = operand1 * operand2
        else:
            raise ValueError(
                f"Помилка! Невідома операція: {operation}")
        # Поміщаємо результат назад на стек
        self._stack.append(result)

    def get_result(self) -> Optional[int]:
        """Повертає значення з вершини стека."""
        if not self._stack:
            return None
        return self._stack[-1]
```

Лістинг 6: Клас `StackCalculator`

Зауважимо, що в процесі виконання команди `eval` може виникати помилка у разі, якщо стек містить менше за два елементи.

Трансляція реалізується методом `Translator.get_commands()`.

Підказка типу для команди стекового обчислювача

`Command = Tuple[str, int, str, None]`

```
class Translator:
    """
    Транслятор, що перетворює AST на послідовність команд
    для стекового обчислювача.
    """
    def __init__(self):
        self.commands: List[Command] = []

    def translate(self, node: Expression):
        """Рекурсивно обходить AST і генерує команди."""
        if isinstance(node, Constant):
            # Генеруємо команду save для константи
            self.commands.append(("save", node.value))
        elif isinstance(node, Plus):
            # Спочатку обробляємо лівий та правий операнди
            self.translate(node.left)
            self.translate(node.right)
            # Потім генеруємо команду eval
            self.commands.append(("eval", "plus"))
        elif isinstance(node, Mult):
            # Спочатку обробляємо лівий та правий операнди
            self.translate(node.left)
            self.translate(node.right)
            # Потім генеруємо команду eval
            self.commands.append(("eval", "mult"))
        else:
            raise TypeError(f"Невідомий тип вузла AST: {type(node)}")

    def get_commands(self) -> List[Command]:
        """Повертає згенерований список команд."""
        return self.commands
```

Лістинг 7: Клас `Translator`

Для отриманого вище абстрактного синтаксичного дерева буде сгенерована наступна послідовність команд.

```
save 3
save 10
eval mult
save 5
eval plus
```

3 Особливості процесу компіляції для предметно-орієнтованих мов

Перш ніж почати відповідати на поставлені вище питання, давайте з'ясуємо чим процес компіляції у випадку предметно-орієнтованих мов відрізняється від процесу компіляції у випадку мов загального призначення. Таке розуміння дає шанс на спрощення компілятора зважаючи на розуміння зазначених особливостей.

Особливості компіляції предметно-орієнтованої мови полягають у її спрощеності та вузькій спрямованості порівняно з мовами загального призначення. Мета компілятора – не створити універсальний машинний код, а перетворити специфічну, виразну логіку предметної області на вже існуючий код або дані.

Спрощеність та вузька спеціалізація мови. Стосується простоти граматики та відсутності складних оптимізацій.

Зазвичай предметно-орієнтована мова має набагато простішу граматику, ніж, наприклад, C++ або Java. Це означає, що етапи лексичного та синтаксичного аналізу є менш складними. Замість обробки сотень ключових слів та складних структур, компілятор працює з обмеженим набором команд, що дозволяє уникнути багатьох проблем, таких як неоднозначність граматики.

Компілятори предметно-орієнтованих мов рідко виконують складні оптимізації коду, оскільки їхнє завдання – швидке та коректне перетворення. Оптимізація часто відкладається на цільову платформу (наприклад, оптимізація SQL-запиту виконується базою даних, а не компілятором відповідної предметно-орієнтованої мови).

Специфічний характер трансляції. Замість генерації машинного або байт-коду, компілятор предметно-орієнтованої мови часто перетворює код на іншу мову високого рівня, наприклад, Python, C++ або JavaScript. Також він може генерувати XML, SQL-запити, або навіть просто виклики методів в існуючій бібліотеці.

Трансляція може бути реалізована через заповнення текстових шаблонів. Наприклад, кожна інструкція предметно-орієнтованої мови може відповідати певному шаблону Python-коду, куди вставляються значення з АСД.

Багато предметно-орієнтованих мов не компілюються, а інтерпретуються. Код такої мови розбирається в АСД, а потім обробляється класом, що реалізує шаблон “відвідувач” (Visitor), який безпосередньо виконує дії, не генеруючи проміжний код. Це притаманно для мов, що описують послідовність дій (наприклад, вже згадана раніше мова Gherkin [6]).

Швидка розробка та інтеграція. Розробники рідко пишуть компілятори для предметно-орієнтованих мов “з нуля”. Вони використовують генератори парсерів, такі як ANTLR [2] або Tree-sitter [1], які автоматично створюють лексичний та синтаксичний аналізатор на основі граматики. Це значно прискорює процес розробки.

Компілятори для предметно-орієнтованих мов тісно інтегруються з існуючим кодом. Вони можуть генерувати код, що залежить від певної структури даних або викликає методи з вже написаної бібліотеки. Це робить мову потужним інструментом для розширення функціональності без необхідності переписувати весь застосунок.

Підсумовуючи, компіляція предметно-орієнтованої мови у порівнянні з компіляцією мови загального призначення є більш *цільеспрямованим* і *ефективним* процесом, що мінімізує складність і зосереджується на перетворенні вузькоспеціалізованої логіки на виконува-

ні інструкції або дані, які вже можуть бути оброблені іншими, більш універсальними інструментами.

4 Висновки

Підсумовуючи викладене, можна сказати, що компілятор є програмою, яка перетворює вихідний код, написаний однією (зазвичай високорівневою) мовою – *вихідна мова*, на код іншою мовою – *цільова мова*, який може бути виконаний комп'ютером.

Компілятор виконує чотири ключові етапи компіляції:

1. Лексичний аналіз (розпізнавання слів) – це перший етап, де вихідний код перетворюється на послідовність "токенів" (значущих слів) за допомогою регулярних виразів та скінченних автоматів, виявляючи лексичні помилки.
2. Синтаксичний аналіз (перевірка відповідності граматиці) – на цьому етапі перевіряється відповідність послідовності токенів граматичним правилам мови. Результатом є побудова абстрактного синтаксичного дерева (АСД), що представляє ієрархічну структуру програми. На цьому етапі виявляються синтаксичні помилки. Для мов зі складною граматикою можуть використовуватися генератори парсерів, а для простих – ліво-рекурсивний спадний парсинг.
3. Семантичний аналіз (перевірка логіки) – цей етап перевіряє логічну коректність програми, включаючи сумісність типів, правильність використання змінних та визначеність функцій. Для предметно-орієнтованих мов семантичний аналіз дозволяє здійснювати валідацію на доменному рівні, що допомагає зменшити логічні помилки, оскільки конструкції мови можуть бути заздалегідь визначені як "безпечні". Повідомлення про помилки, виявлені на цьому етапі, мають бути зрозумілими для доменних експертів.
4. Трансляція (генерація цільового коду) – це останній етап, метою якого є генерація виконуваного цільового коду (низькорівневого, машинного, байт-коду або коду іншої мови програмування). У контексті предметно-орієнтованих мов це може включати синтез застосунків з програмних компонентів.

Етапи компіляції взаємопов'язані через структури даних, такі як послідовності токенів та абстрактні синтаксичні дерева, і керуються правилами розпізнавання, граматичними правилами, семантичними обмеженнями та правилами трансляції. Теорія формальних мов є фундаментальною для відповіді на питання щодо представлення та використання цих правил.

Процес компіляції для предметно-орієнтованих мов відрізняється від компіляції мов загального призначення, оскільки він є більш цілеспрямованим та ефективним:

- Предметно-орієнтовані мови мають значно простішу граматику, що спрощує етапи лексичного та синтаксичного аналізу. Компілятори для таких мов рідко виконують складні оптимізації, оскільки це часто делегується цільовій платформі (наприклад, оптимізація SQL-запиту виконується СУБД).
- Замість генерації низькорівневого коду, компілятори предметно-орієнтованих мов часто перетворюють код на іншу високорівневу мову (наприклад, Python, C++, JavaScript), XML, SQL-запити або просто виклики методів в існуючих бібліотеках. Деякі мови інтерпретуються безпосередньо через обробку АСД.

- Компілятори для предметно-орієнтованих мов рідко створюються “з нуля”, часто використовуються генератори парсерів (наприклад, ANTLR, Tree-sitter), що значно прискорює розробку. Вони тісно інтегруються з існуючим кодом, генеруючи залежний від нього код.

Таким чином, розуміння етапів компіляції та їх адаптація до специфіки предметно-орієнтованих мов є ключовим для створення ефективних та цілеспрямованих інструментів розробки, які мінімізують складність і зосереджуються на перетворенні вузькоспеціалізованої логіки на виконувані інструкції або дані.

Література

1. *Tree-sitter*. Tree-sitter. — 2025. — Дата доступу: 07.08.2025. <https://tree-sitter.github.io/tree-sitter/>.
2. *Terence Parr and The ANTLR Project Team*. ANTLR (ANother Tool for Language Recognition). — 2025. — Дата доступу: 07.08.2025. <https://www.antlr.org/>.
3. *Van Rossum, Guido*. PEG Parsers. — 02.2021. — URL: https://medium.com/@gvanrossum_83706/peg-parsers-7ed72462f97c ; Дата доступу: 07.08.2025. Medium.
4. *Ullman, J.* Implementation of logical query languages for databases // ACM Transactions on Database Systems. — 1985. — Т. 10, № 3. — С. 289—321.
5. *aCode*. Уроки Python. Регулярні вирази в Python. — 2025. — Дата доступу: 10.08.2025. <https://acode.com.ua/regex-python/>.
6. *Gherkin UFT*. Gherkin. — 2025. — Дата доступу: 07.08.2025. <https://www.gherkinuft.com/gherkin/>.